

LLM do Zero

Hospedagem Local de Modelos LLM via Ollama

Principais conceitos, exemplos, APIs, parâmetros, quantização, monitoramento e boas práticas.

Armando Soares Sousa



Objetivos de aprendizagem

Ao final da unidade, o aluno deve conseguir instalar, operar e integrar modelos locais com disciplina experimental.

- Instalar e validar o Ollama em Linux, macOS, Windows/WSL e Docker.
- Gerenciar modelos, tags, sessões ativas e registros de experimentos.
- Controlar inferência com parâmetros por requisição e via Modelfile.
- Usar a API nativa e a API compatível com OpenAI em aplicações Python.
- Avaliar limitações de CPU, RAM, VRAM, quantização e monitoramento.

A ideia central: inferência local é excelente para aprendizado, prototipação e validação, mas exige controle operacional.

Unidades

As seguintes unidades serão trabalhadas neste curso introdutório sobre o Ollama

Unidade	Tema central	Ampliação nesta versão
1	Instalação e configuração	Linux, macOS, Windows, Docker, variáveis e diagnóstico
2	Gerenciamento de modelos	CLI, tags, sessões, matriz de experimentos e reprodutibilidade
3	Parâmetros e prompts	Options, Modelfile, templates e exemplos de prompt técnico
4	Ollama como serviço	API nativa, OpenAI-like, streaming, vision, embeddings e Python
5	Hardware e boas práticas	CPU, RAM, VRAM, quantização, sintomas, monitoramento e decisão de uso

Cada unidade tem uma prática executável.

Instalação e Configuração

Preparar o ambiente local, entender onde os modelos ficam armazenados e validar o serviço antes de iniciar testes.



Por que hospedar LLMs localmente?

Ollama facilita baixar, executar e expor modelos abertos em uma máquina de desenvolvimento.

- Privacidade: dados de teste podem permanecer no ambiente local.
- Baixo atrito: CLI simples, serviço HTTP e modelos pré-empacotados.
- Custo previsível: sem cobrança por token durante experimentos locais.
- Controle técnico: tags, parâmetros, Modelfile e observação do hardware.
- Limite importante: local não substitui arquitetura de produção escalável.

Use local para aprender, testar e validar; use vLLM/gateways/clusters quando precisar de escala.

Instalação: visão geral por sistema

A escolha do caminho depende do sistema operacional, GPU disponível e objetivo do laboratório.

- Linux: instalação por script oficial e execução como serviço do sistema.
- macOS: aplicativo desktop e bom suporte a Apple Silicon.
- Windows: instalador nativo ou WSL2 para fluxo mais próximo do Linux.
- Docker: útil para isolamento, automação e ambientes reproduzíveis.
- Após instalar: valide CLI, API, diretório de modelos e consumo de recursos.



```
# Linux/macOS: validar CLI
ollama --version
ollama list

# API local padrão
curl http://localhost:11434/api/tags
```

Instale primeiro, valide depois; não comece testes de modelo antes de confirmar o serviço.

Instalação no Linux

No Linux, o Ollama costuma ser instalado como serviço, facilitando inicialização automática e uso por scripts.

- Execute o instalador oficial e confirme se o binário ficou disponível.
- Verifique se o serviço está ativo antes de baixar modelos grandes.
- Se usar GPU NVIDIA, valide drivers e CUDA com `nvidia-smi`.
- Em servidores, controle quem pode acessar a porta 11434.
- Documente distribuição, kernel, CPU, RAM e GPU do laboratório.



```
curl -fsSL https://ollama.com/install.sh | sh
```

```
ollama --version  
systemctl status ollama
```

```
# Teste simples  
ollama run llama3.2:3b
```

Evite expor o serviço para a rede sem firewall, proxy ou controle de acesso.

Instalação no macOS

No macOS, o uso típico é iniciar pelo aplicativo e depois operar via terminal.

- Baixe e instale o aplicativo do Ollama para macOS.
- Apple Silicon usa memória unificada; o limite prático depende da RAM total.
- Use modelos pequenos para aulas e testes rápidos.
- O serviço local fica acessível para a CLI e para chamadas HTTP.
- Monitore Activity Monitor para memória, CPU e temperatura.



```
ollama --version
ollama pull qwen2.5:3b
ollama run qwen2.5:3b

# Encerrar sessão ativa
ollama ps
ollama stop qwen2.5:3b
```

Em notebooks, carga prolongada pode causar thermal throttling e reduzir tokens por segundo.

Windows, WSL2 e Docker

O Windows pode ser usado diretamente, mas WSL2 e Docker ajudam quando o laboratório exige scripts Linux.

- Instalador nativo: caminho mais simples para iniciantes.
- WSL2: bom para aulas com shell, Python e ferramentas Linux.
- Docker: ajuda a isolar versões e volumes de modelos.
- Com GPU, valide suporte do runtime do container.
- Padronize comandos para reduzir variações entre máquinas.



```
# Docker com GPU NVIDIA
docker run -d --gpus=all \
-v ollama:/root/.ollama \
-p 11434:11434 \
--name ollama ollama/ollama
```

Docker simplifica reprodução, mas não elimina a necessidade de configurar GPU, volume e rede.

Validação da instalação

Antes de baixar modelos grandes, faça testes pequenos e objetivos.

- CLI: verificar versão, lista local e execução de um modelo leve.
- API: consultar /api/tags para confirmar serviço HTTP.
- Rede: confirmar que o host correto está aceitando conexões.
- Hardware: medir CPU, RAM e VRAM durante uma resposta simples.
- Logs: salvar erro, comando executado e ambiente usado.



```
ollama --version
ollama list
ollama pull llama3.2:3b

curl http://localhost:11434/api/tags

# Linux
htop
free -h
nvidia-smi
```

Validação mínima: CLI funciona, API responde e pelo menos um modelo pequeno executa.

Sistema de arquivos

Os modelos ocupam espaço significativo e devem ser tratados como ativos de laboratório.

- O diretório de modelos pode ser alterado via OLLAMA_MODELS.
- Use disco rápido para reduzir tempo de carregamento inicial.
- Evite duplicar modelos grandes em cada projeto ou usuário.
- Planeje limpeza periódica com ollama rm.
- Inclua o diretório de modelos no inventário do ambiente.



```
# Listar modelos locais  
ollama list
```

```
# Remover modelo não usado  
ollama rm nome:tag
```

```
# Usar outro diretório  
export OLLAMA_MODELS=/dados/ollama-models
```

Modelos são dependências pesadas: trate versões e armazenamento como parte do experimento.

Variáveis de ambiente úteis

Variáveis controlam localização dos modelos, host de escuta e comportamento do serviço.

- OLLAMA_HOST: define endereço e porta do serviço.
- OLLAMA_MODELS: muda o local de armazenamento dos modelos.
- OLLAMA_KEEP_ALIVE: controla quanto tempo o modelo fica carregado.
- OLLAMA_NUM_PARALLEL: ajusta paralelismo conforme ambiente.
- Registre variáveis usadas em cada laboratório.



```
export OLLAMA_HOST=0.0.0.0:11434
export OLLAMA_MODELS=/srv/ollama/models
export OLLAMA_KEEP_ALIVE=10m
```

```
# Reinicie o serviço após alterar variáveis
sudo systemctl restart ollama
```

Atenção: OLLAMA_HOST=0.0.0.0 expõe o serviço na rede; use somente com controles adequados.

Primeiro modelo: baixar e executar

Comece com modelos pequenos para validar instalação, latência e consumo de memória.

- Escolha uma tag explícita, evitando depender de latest em experimentos.
- Teste perguntas curtas e respostas com limite de tokens.
- Compare tempo de carregamento inicial versus respostas subsequentes.
- Registre sintomas: lentidão, OOM, travamentos ou aquecimento.
- Só avance para modelos maiores após estabilizar o fluxo.



```
ollama pull llama3.2:3b  
ollama run llama3.2:3b
```

```
>>> Explique em 5 linhas o que é RAG.
```

```
# Ver modelos carregados  
ollama ps
```

Boa prática de aula: todos começam com o mesmo modelo pequeno e depois comparam alternativas.

Diagnóstico rápido de instalação

Problemas comuns geralmente aparecem como erro de rede, modelo ausente ou falta de recursos.

Connection refused: serviço não está ativo ou porta incorreta.

Model not found: modelo/tag não foi baixado ou nome está incorreto.

Out of memory: modelo grande demais para RAM/VRAM disponível.

Resposta lenta: CPU-only, disco lento, modelo grande ou contexto alto.

Permissão: diretório de modelos sem acesso de leitura/escrita.



```
# Serviço responde?  
curl http://localhost:11434/api/tags
```

```
# Modelo existe localmente?  
ollama list
```

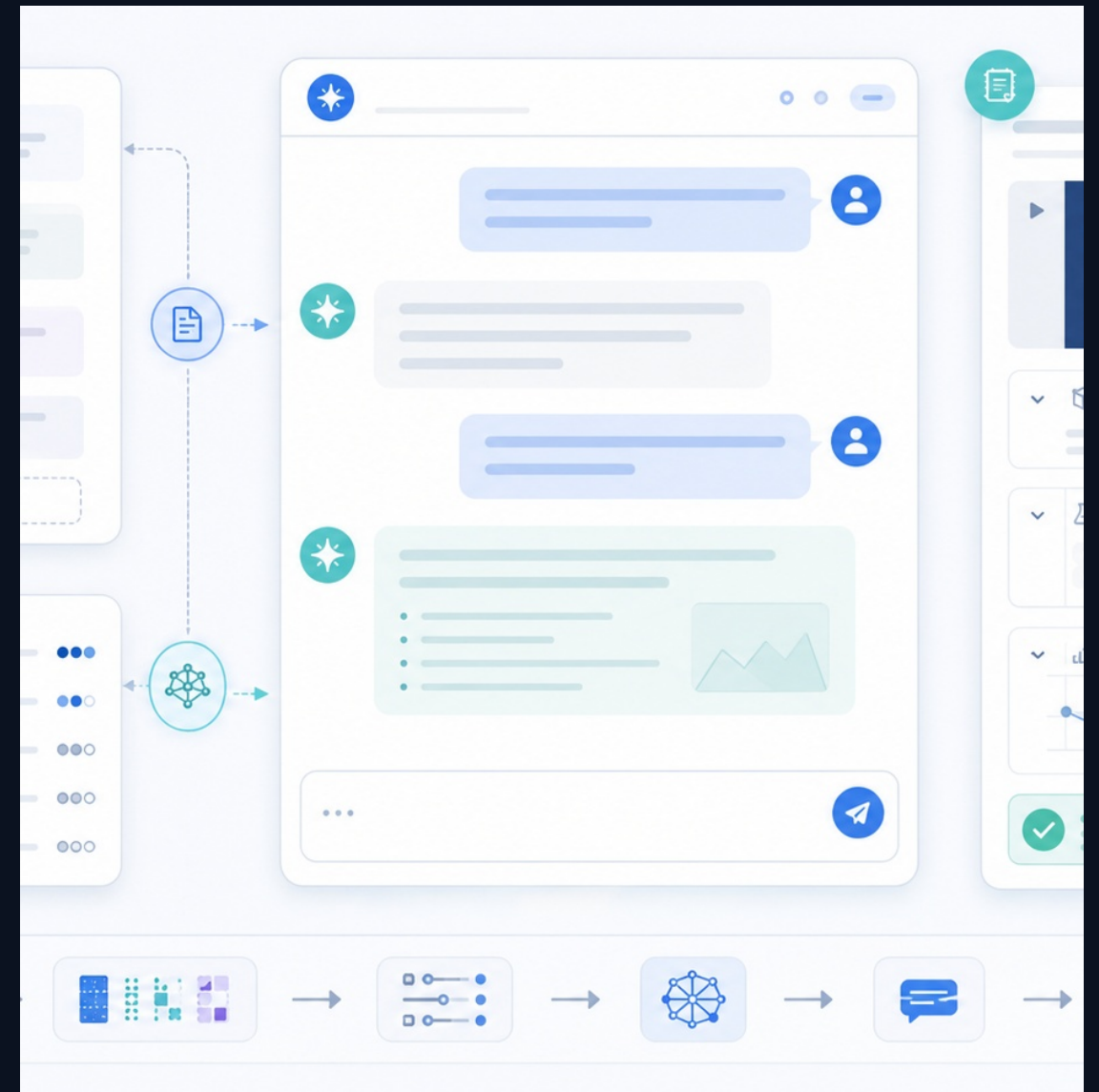
```
# Sessões carregadas?  
ollama ps
```

```
# Ver logs do serviço  
journalctl -u ollama -n 80
```

Sempre diagnostique do mais simples para o mais caro: serviço, modelo, recursos, rede e logs.

Gerenciamento e Execução de Modelos

Operar modelos com comandos simples, sessões controladas e documentação suficiente para reproduzir resultados.



Operações mais comuns

A CLI do Ollama cobre o ciclo básico de baixar, executar, inspecionar, copiar e remover modelos.

- pull: baixa um modelo/tag do repositório.
- run: executa o modelo em modo interativo.
- list: mostra modelos instalados localmente.
- show: exibe metadados do modelo.
- ps/stop/rm: monitora, encerra sessão ou remove modelo.

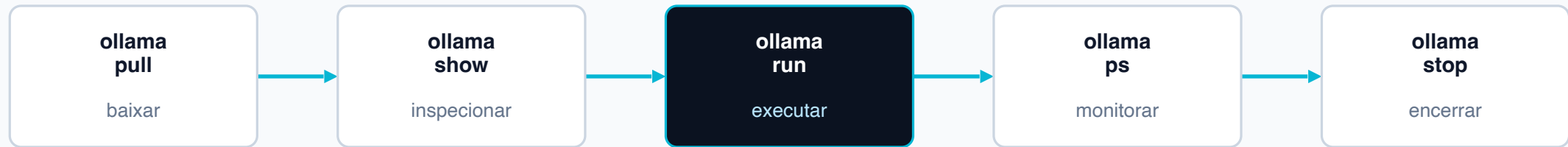


```
ollama pull qwen2.5:7b
ollama run qwen2.5:7b
ollama list
ollama show qwen2.5:7b
ollama ps
ollama stop qwen2.5:7b
ollama rm qwen2.5:7b
```

Comandos simples ficam poderosos quando acompanhados de registro experimental.

Ciclo de vida do modelo

Do download à execução, o fluxo deve ser simples, rastreável e reversível.



Registre sempre: nome do modelo + tag + parâmetros + data + hardware usado no teste.

Controle de sessão de um modelo ativo

Modelos podem permanecer carregados para acelerar chamadas seguintes, consumindo RAM/VRAM.

- Use `ollama ps` para ver modelos carregados e tempo de permanência.
- Use `ollama stop` para liberar memória após testes.
- `keep_alive` evita recarregamento em cenários interativos.
- Em máquinas compartilhadas, finalize sessões após o laboratório.
- Registre quando a primeira chamada inclui tempo de carregamento.

```
ollama ps
ollama stop llama3.2:3b

# Requisição com keep_alive
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2:3b",
  "messages": [{"role": "user", "content": "Oi"}],
  "keep_alive": "10m"
}'
```

Sessão ativa acelera, mas também prende memória. Controle é parte do experimento.

Modelos mais comuns

Escolha o modelo pela tarefa, idioma, tamanho, licença e hardware disponível.

Família	Uso típico	Observação para laboratório
Llama	chat geral, raciocínio e protótipos	bom ponto de partida em versões pequenas e médias
Gemma	chat geral e uso eficiente	útil para comparar qualidade x tamanho
Qwen	código, ferramentas e modelos multilíngues	boa opção para tarefas técnicas e exemplos de programação
Mistral	chat leve e inferência eficiente	boa alternativa para hardware limitado
nomic-embed-text	embeddings para busca semântica	base simples para RAG local

Evite comparar modelos sem fixar prompt, parâmetros, tag e hardware.

Tags e versões para reprodutibilidade

A tag identifica uma variante do modelo; sem tag fixa, o experimento fica difícil de repetir.

- Prefira tags explícitas em aulas e relatórios.
- Evite usar latest em comparações formais.
- Documente quantização, tamanho e data do download.
- Use `ollama show` para registrar metadados relevantes.
- Inclua o prompt e parâmetros junto com o nome do modelo.



```
# Menos reprodutível  
ollama run llama3
```

```
# Mais reprodutível  
ollama run llama3.2:3b
```

```
# Metadados  
ollama show llama3.2:3b
```

Reprodutibilidade mínima = modelo:tag + prompt + parâmetros + hardware + data.

Documentar versões e configurações

Resultados de LLM local dependem do ambiente. Sem log, não há comparação confiável.

- Modelo e tag usados.
- Sistema operacional e versão do Ollama.
- CPU, RAM, GPU/VRAM e tipo de disco.
- Parâmetros de inferência e tamanho do contexto.
- Métricas: tempo de carregamento, latência, tokens/segundo e sintomas.



```
experimento: resumo_tecnico_01  
data: 2026-05-17  
ollama: 0.x  
modelo: llama3.2:3b  
hardware: 16GB RAM, sem GPU  
opcoes: temperature=0.2, num_ctx=4096  
resultado: resposta ok, latencia alta
```

Logs simples em Markdown já melhoram muito a qualidade dos experimentos.

Exemplo: matriz de experimentos

Use uma pequena matriz para comparar qualidade, custo local e comportamento dos modelos.

- Fixe a mesma tarefa para todos os modelos.
- Altere apenas um fator por vez: modelo, quantização ou parâmetro.
- Registre tempo total, tamanho da resposta e qualidade percebida.
- Inclua falhas, travamentos ou respostas incompletas.
- Transforme resultados em tabela para discussão em sala.

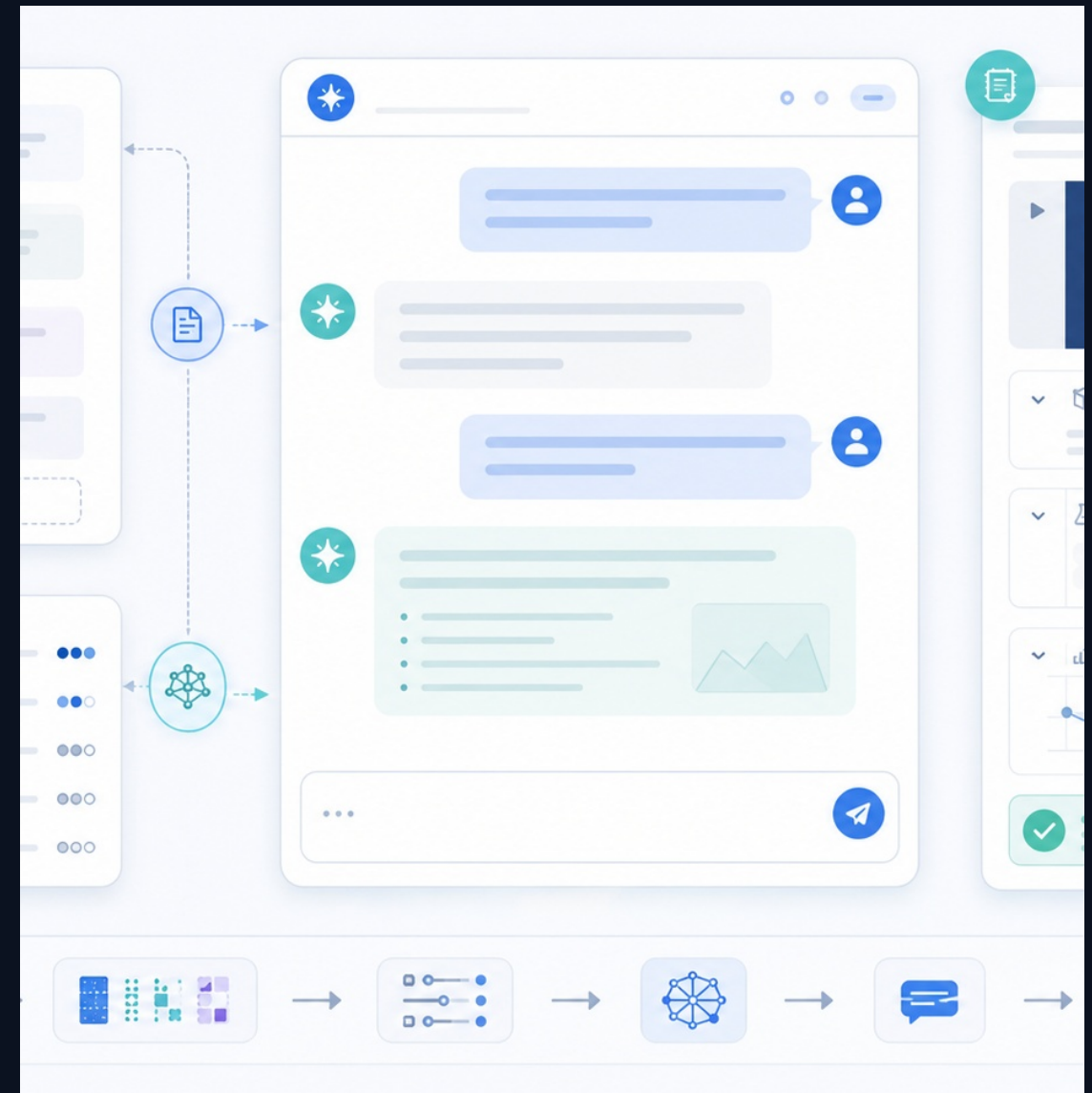


Modelo	Temp	Contexto	Tempo	Qualidade	
--- ---	---	---	---	---	
llama3.2:3b	0.2	4096	18s	boa	
qwen2.5:7b	0.2	4096	42s	melhor	
mistral:7b	0.7	4096	39s	criativa	

Comparar LLM sem matriz vira impressão subjetiva; com matriz vira evidência.

Parâmetros de Inferência e Prompt Engineering

Controlar o comportamento do modelo por requisição, por Modelfile e por prompts padronizados.



Controle do comportamento do modelo

O comportamento pode ser ajustado no momento da chamada ou encapsulado em um modelo customizado.

- Por requisição: rápido para testes e A/B de parâmetros.
- Por Modelfile: útil para criar assistentes reutilizáveis.
- Prompt: define papel, tarefa, contexto, formato e restrições.
- Parâmetros: controlam criatividade, diversidade e tamanho da resposta.
- Reprodutibilidade: salve tudo que influencia a saída.

Pense em três camadas: prompt define intenção; parâmetros definem estilo; Modelfile empacota padrão.

Parâmetros por requisição

Options permitem experimentar sem criar outro modelo.

- temperature: aumenta ou reduz variabilidade da resposta.
- top_p/top_k: controlam amostragem e diversidade lexical.
- num_ctx: define janela de contexto disponível para entrada.
- num_predict: limita quantidade de tokens gerados.
- stop: define padrões de parada para respostas controladas.

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2:3b",
  "messages": [{"role": "user", "content": "Explique RAG"}],
  "options": {
    "temperature": 0.2,
    "top_p": 0.9,
    "num_ctx": 4096,
    "num_predict": 250
  }
}'
```

Para respostas técnicas, comece com temperature baixa e limite explícito de saída.

Parâmetros via Modelfile

O Modelfile cria um novo modelo local com instruções e opções pré-definidas.

- FROM: modelo base usado como ponto de partida.
- SYSTEM: comportamento e persona do assistente.
- PARAMETER: valores-padrão para inferência.
- TEMPLATE: formato de conversa, quando necessário.
- ollama create: constrói o modelo customizado.



```
FROM llama3.2:3b
```

```
SYSTEM """
```

```
Você é um tutor de IA Generativa.
```

```
Explique com clareza e exemplos curtos.
```

```
"""
```

```
PARAMETER temperature 0.2
```

```
PARAMETER num_ctx 4096
```

Modelfile é útil quando uma turma ou equipe precisa repetir o mesmo padrão de resposta.

Criando um assistente customizado

Exemplo de empacotamento de um tutor local para aulas práticas.

- Escreva o arquivo Modelfile com instrução do sistema.
- Crie o modelo local com nome semântico.
- Execute com ollama run como qualquer outro modelo.
- Teste com perguntas-padrão da disciplina.
- Versione o Modelfile junto do material da aula.



```
# Criar modelo customizado
ollama create tutor-llm-zero -f Modelfile

# Executar
ollama run tutor-llm-zero

>>> Crie um exemplo de prompt para RAG.
```

O modelo customizado não treina pesos; ele empacota instruções e parâmetros sobre um modelo base.

Estrutura básica de um prompt

Um bom prompt separa intenção, contexto, tarefa, formato e restrições.

Papel

Você é um assistente técnico...

Tarefa

Explique o conceito...

Contexto

Considere alunos do último período...

Formato

Responda em tabela e exemplos...

Restrições

Use termos simples e não invente dados...

Quanto mais reprodutível for o prompt, mais fácil comparar modelos, tags e parâmetros.

Criação de prompts reutilizáveis

Prompts reutilizáveis reduzem improviso, melhoram comparação e ajudam na avaliação.

- Separe variáveis do texto fixo do prompt.
- Inclua critérios de qualidade esperados.
- Padronize formato de resposta para facilitar análise.
- Evite instruções ambíguas ou contraditórias.
- Mantenha versões dos prompts em arquivos Markdown ou YAML.

```
nome: resumo_tecnico
versao: 1.0
variaveis:
  dominio: "LLMs locais"
  publico: "alunos de computacao"
saida:
  formato: "tabela + exemplo"
critérios:
  - clareza
  - precisão
  - exemplos práticos
```

Prompt reutilizável é artefato de engenharia, não apenas texto improvisado.

Prompt como template parametrizável

Templates permitem trocar domínio, tarefa, formato e restrições sem reescrever tudo.

Contexto: informa domínio, público e material de referência.

Tarefa: descreve exatamente o que deve ser produzido.

Formato: define estrutura de saída esperada.

Restrições: limita tamanho, tom, fonte e estilo.

Critérios: explicita o que torna a resposta aceitável.



```
Contexto: {dominio}  
Público: {publico}  
Tarefa: {instrucao}  
Formato de resposta: {formato}  
Restrições: {regras}  
Critérios de qualidade: {criterios}
```

Produza a resposta final em português claro.

Quanto mais padronizado o template, mais fácil automatizar testes e revisão.

Exemplo de prompt para resumo técnico

Um exemplo pronto para usar em laboratório com modelos locais.

- Defina o papel do modelo como tutor técnico.
- Forneça escopo e público-alvo.
- Peça uma saída estruturada e verificável.
- Limite tamanho para reduzir latência.
- Inclua solicitação explícita de exemplos.



Você é um professor de Ciência da Computação.
Explique o que é quantização em LLMs para alunos
do último período.

Responda em 3 partes:

1. definição curta;
2. tabela com FP16, INT8 e INT4;
3. exemplo de quando usar quantização.

Use até 250 palavras.

Esse prompt é pequeno, avaliável e adequado para comparar modelos.

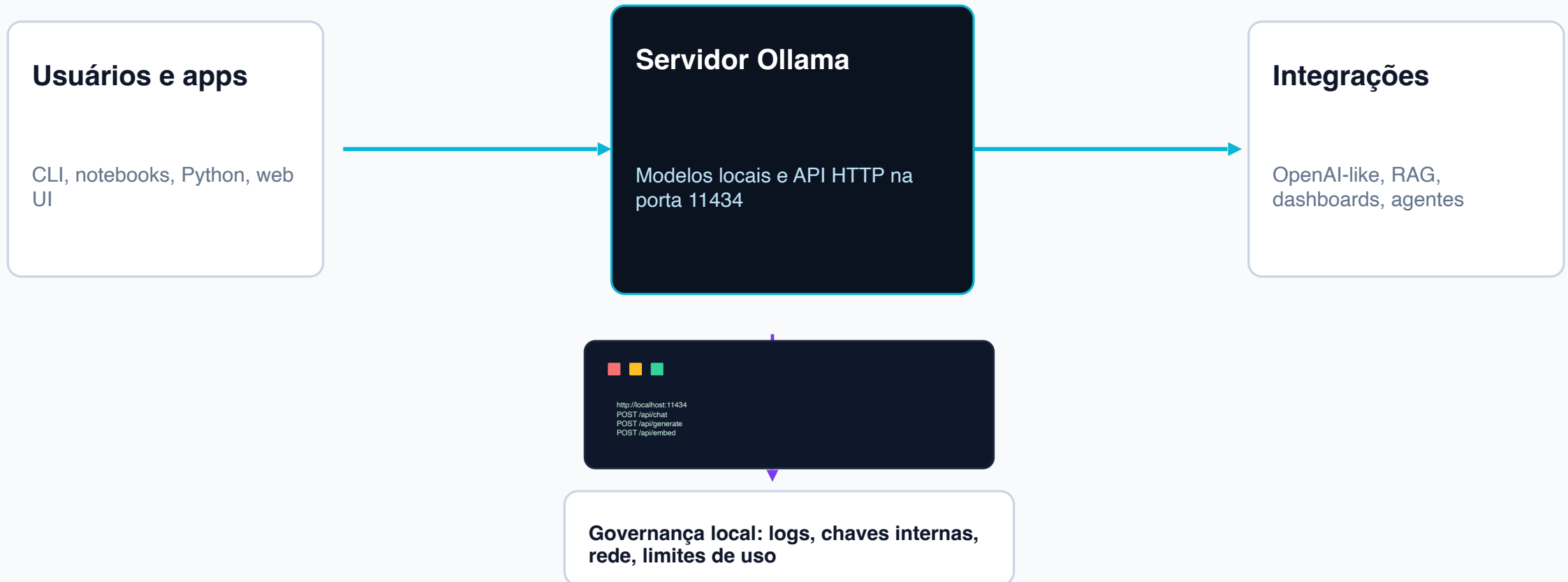
Usando o Ollama como um Serviço

Transformar o modelo local em uma API para scripts, aplicações, interfaces e pipelines de RAG.



Ollama como serviço local

A API fica publicada localmente e pode atender scripts, notebooks, protótipos e interfaces internas.



API nativa: /api/chat e /api/generate

A API nativa do Ollama é direta e oferece controle de mensagens, opções, streaming e ciclo de vida.

- /api/chat: formato de conversa com lista de mensagens.
- /api/generate: geração por prompt único.
- /api/tags: lista modelos locais disponíveis.
- stream pode ser true para respostas incrementais.
- options permite passar parâmetros de inferência.

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2:3b",
  "messages": [
    {"role": "system", "content": "Seja objetivo."},
    {"role": "user", "content": "Explique embeddings."}
  ],
  "stream": false
}'
```

Use /api/chat para aplicações conversacionais e /api/generate para tarefas simples de geração.

API compatível com OpenAI

A compatibilidade facilita reaproveitar clientes, exemplos e ferramentas que já usam o padrão OpenAI.

- Base URL local aponta para o serviço do Ollama.
- Cliente OpenAI pode chamar modelos locais com poucas mudanças.
- Útil para protótipos que podem migrar entre local e nuvem.
- Nem todos os recursos das APIs comerciais são equivalentes.
- Valide campos suportados antes de padronizar a integração.

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"
)

resp = client.chat.completions.create(
    model="llama3.2:3b",
    messages=[{"role": "user", "content": "Explique RAG"}]
)
print(resp.choices[0].message.content)
```

Compatibilidade reduz acoplamento: mesma aplicação pode testar modelo local e modelo remoto.

Streaming: experiência em tempo real

Streaming melhora a percepção de latência e permite interfaces de chat mais responsivas.

- Com `stream=true`, a resposta chega em pequenos blocos.
- Bom para chat, copilotos e demonstrações ao vivo.
- Desative streaming para jobs batch ou avaliação offline.
- A aplicação precisa concatenar os fragmentos recebidos.
- Mensure TTFT e tempo total separadamente.

```
import ollama

stream = ollama.chat(
    model='llama3.2:3b',
    messages=[{'role':'user','content':'Crie um resumo'}],
    stream=True
)

for chunk in stream:
    print(chunk['message']['content'], end="")
```

Streaming não torna o modelo mais rápido, mas melhora a experiência do usuário.

Vision: análise de imagens local

Modelos multimodais permitem descrever ou analisar imagens no ambiente local.

- Use modelos com capacidade de visão, quando disponíveis.
- Útil para OCR visual simples, inspeção de diagramas e protótipos.
- Não confunda análise visual com OCR robusto ou perícia técnica.
- Teste limitações: resolução, detalhes pequenos e alucinação visual.
- Registre imagem, modelo, prompt e resposta para auditoria.

```
import ollama

resp = ollama.chat(
    model='llava:7b',
    messages=[{
        'role':'user',
        'content':'Descreva os elementos principais da imagem.',
        'images':['diagrama.png']
    }]
)
print(resp['message']['content'])
```

Para ensino, vision é excelente para discutir limitações de interpretação multimodal.

Embedding: vetores de texto para RAG

Embeddings transformam texto em vetores para busca semântica e recuperação de contexto.

- Use um modelo específico de embedding, como nomic-embed-text.
- Armazene vetores em base local ou vector store.
- RAG local combina: documentos, embeddings, busca e chat.
- Embeddings não geram resposta; eles recuperam trechos relevantes.
- Avalie qualidade por consultas reais e exemplos de erro.

```
curl http://localhost:11434/api/embed -d '{
  "model": "nomic-embed-text",
  "input": [
    "Ollama executa modelos localmente",
    "RAG combina busca e geração"
  ]
}'
```

RAG local é uma boa ponte entre aula de LLM, engenharia de dados e sistemas de informação.

Ollama nativo vs. OpenAI-like

Duas superfícies de API, duas estratégias de integração.

Recurso	Ollama nativo	OpenAI-like
Streaming	Direto com biblioteca ollama ou /api/chat	Compatível com clientes OpenAI
Vision	Mensagens com campo images	Depende do suporte do cliente e endpoint
Embedding	/api/embed e biblioteca nativa	Pode ser adaptado para clientes compatíveis
Migração	Mais específico ao Ollama	Menor mudança para código já existente
Controle	Mais próximo dos recursos Ollama	Mais portátil, mas nem sempre completo

Regra simples: use nativo para explorar Ollama; use OpenAI-like para reaproveitar aplicações.

Arquitetura local com governança mínima

Mesmo em laboratório, é útil criar limites e padrões para evitar uso desorganizado.

- Defina quem pode acessar o serviço local e de onde.
- Use proxy ou túnel apenas quando necessário e com controle.
- Padronize modelos, tags e parâmetros por prática.
- Registre métricas básicas de uso e erros.
- Separe protótipos locais de serviços de produção.



```
app.py -> OpenAI client -> localhost:11434/v1  
rag.py -> /api/embed + vector store local  
chat UI -> /api/chat streaming  
logs/ -> prompts, parametros, tempos  
models -> OLLAMA_MODELS
```

API local sem governança vira dependência invisível: documente e limite desde o início.

Limitações de Hardware e Boas Práticas

Entender CPU, RAM, VRAM, quantização e sintomas de saturação antes de decidir se local é suficiente.



Recursos computacionais envolvidos

O gargalo de inferência muda conforme modelo, quantização, contexto e tipo de hardware.

CPU

Orquestração e inferência sem GPU

latência maior

RAM

Pesos, buffers e contexto

OOM se insuficiente

VRAM

Operações matriciais na GPU

principal gargalo

Disco

Download, cache e carregamento

I/O lento atrasa

Regra prática: o recurso limitante define tamanho do modelo, velocidade e estabilidade.

Uso de CPU na inferência local

A CPU permite executar modelos sem GPU, mas a latência pode ser alta.

- Cenários típicos: sem GPU, modelos pequenos, testes de baixa taxa.
- Quanto maior o modelo, maior o tempo por token.
- Múltiplos núcleos ajudam, mas não substituem VRAM para modelos grandes.
- Útil para ensino, scripts simples e validação inicial.
- Não é indicado para muitos usuários simultâneos.



```
# Monitorar CPU no Linux  
htop
```

```
# Coletar info básica  
lscpu
```

```
# Teste controlado  
time ollama run llama3.2:3b "Explique CPU-only"
```

Inferência em CPU é funcional, mas raramente é escalável para uso interativo intenso.

Uso de RAM na inferência local

RAM insuficiente causa lentidão extrema, swap e erros de memória.

- Pesos e buffers podem ficar em RAM quando não cabem na GPU.
- Contextos longos aumentam o consumo de memória.
- Swap em disco pode tornar a inferência impraticável.
- Modelos 7B quantizados são o ponto de partida mais seguro.
- Modelos grandes exigem planejamento antes do download.



```
# Ver memória disponível  
free -h
```

```
# macOS  
vm_stat
```

```
# Sintoma típico  
Killed / OOM / swap alto / resposta muito lenta
```

Sem RAM suficiente, o problema deixa de ser qualidade do modelo e vira estabilidade do sistema.

Uso de VRAM e GPU

A VRAM é o recurso mais crítico para baixa latência em modelos médios ou grandes.

- VRAM armazena pesos e acelera operações matriciais.
- Se o modelo não cabe, parte da execução pode cair para CPU/RAM.
- Monitore memória usada, processos ativos e temperatura.
- Reserve margem para contexto, sistema e outros processos.
- Em laboratório, use um modelo por vez para reduzir disputa por VRAM.



```
# NVIDIA  
nvidia-smi  
  
# Observar  
# - memory.used / memory.total  
# - utilization.gpu  
# - temperature.gpu  
# - processos ativos
```

VRAM livre no início não garante estabilidade durante respostas longas ou contextos maiores.

Impacto da quantização

Quantização reduz precisão dos pesos para diminuir memória e permitir execução local.

- FP16: maior qualidade e maior consumo de memória.
- INT8: boa relação entre qualidade e eficiência.
- INT4/Q4: máxima economia, com possível perda de qualidade.
- Menos bits reduzem RAM/VRAM e podem melhorar velocidade.
- A perda depende da tarefa, do modelo e da quantização usada.

Quantização é uma troca: mais acessibilidade de hardware em troca de possível perda de precisão.

Trade-offs da quantização

A decisão depende do objetivo: prototipagem, qualidade, latência ou custo local.

Benefício	Custo possível	Quando aceitar
Menor RAM/VRAM	Perda leve de qualidade	aulas, protótipos e testes rápidos
Modelos maiores em hardware modesto	Mais erros em tarefas sensíveis	exploração inicial de alternativas
Mais sessões possíveis	Instabilidade em respostas longas	demonstrações controladas
Menor tempo de carregamento	Menos fidelidade matemática	resumos, chat geral e tarefas simples

Sempre teste a tarefa real: a mesma quantização pode ser ótima em resumo e ruim em código.

Quantização no Ollama

Muitos modelos já são distribuídos em variantes prontas para execução local.

- Na maioria dos casos, o usuário não precisa quantizar manualmente.
- Escolha tags adequadas ao hardware e à tarefa.
- Compare tamanho em disco, consumo de memória e qualidade.
- Para embeddings, use modelos específicos, não modelos de chat.
- Evite supor que modelo maior sempre será melhor no seu ambiente.



```
# Exemplos para começar
ollama pull llama3.2:3b
ollama pull qwen2.5-coder:7b
ollama pull mistral:7b
ollama pull nomic-embed-text

ollama list
```

O melhor modelo local é o que cabe, responde no tempo aceitável e resolve a tarefa.

Limitações comuns em ambientes locais

Ambientes locais são excelentes para aprendizagem, mas têm limites técnicos e operacionais.

- Pouca RAM, VRAM insuficiente ou CPU com poucos núcleos.
- Disco lento aumentando carregamento de modelos.
- Ausência de autenticação, quotas, logs e monitoramento real.
- Baixa capacidade para muitos usuários simultâneos.
- Dificuldade de atender SLA rígido de latência.

Ambiente local não é sinônimo de produção; é laboratório controlado e protótipo rápido.

Sintomas comuns de saturação

Os sintomas de hardware aparecem antes de qualquer diagnóstico sofisticado.

- Travamentos do sistema ou interface congelando.
- Processo finalizado pelo sistema operacional.
- Latência excessiva ou resposta token a token muito lenta.
- Respostas incompletas ou interrupções durante geração.
- Aquecimento, ventoinha intensa e queda de desempenho.



```
# Checklist de sintomas  
[ ] CPU 100% por longos períodos  
[ ] RAM/VRAM perto do limite  
[ ] Swap alto  
[ ] Temperatura elevada  
[ ] Modelo reinicia ou descarrega sempre
```

Saturação não se corrige com prompt; reduza modelo, contexto, saída ou carga concorrente.

Boas práticas: escolha de modelo

A escolha deve começar pelo hardware disponível e pela tarefa, não pelo hype do modelo.

- Comece com modelos pequenos, como 3B ou 7B.
- Use versões quantizadas para máquinas comuns.
- Prefira modelos especializados quando a tarefa for específica.
- Teste antes de escalar para modelos maiores.
- Evite modelos grandes sem margem de RAM/VRAM.

Para uma aula prática, confiabilidade vale mais que usar o maior modelo disponível.

Boas práticas: execução e uso

Disciplina operacional evita resultados inconsistentes e problemas de desempenho.

- Feche aplicações pesadas durante testes.
- Use um modelo por vez em máquinas com poucos recursos.
- Limite tokens de saída e tamanho do contexto.
- Monitore CPU, RAM, VRAM e temperatura continuamente.
- Documente comando, modelo, prompt, parâmetros e resultado.



```
# Limitar saída por requisição
"options": {
  "num_predict": 250,
  "temperature": 0.2
}
```

```
# Liberar memória após aula
ollama ps
ollama stop nome:tag
```

LLM local exige disciplina semelhante a experimento científico: controle variáveis e registre evidências.

Boas práticas: desenvolvimento

Durante desenvolvimento, reduza custo local e acelere feedback.

- Use prompts curtos até estabilizar o fluxo.
- Teste etapas isoladas antes de montar pipeline completo.
- Use respostas curtas para depurar integração.
- Separe teste de funcionalidade de teste de qualidade.
- Use local para prototipação, aprendizagem e validação inicial.



```
# Exemplo de prompt de debug  
"Responda apenas OK se recebeu esta mensagem."  
  
# Depois evolua para:  
"Resuma o texto em 5 tópicos e aponte riscos."
```

Antes de avaliar inteligência, avalie se a integração funciona.

Quando não usar inferência local

Há cenários em que um servidor local simples não é a arquitetura correta.

- Muitos usuários simultâneos ou alto volume de requisições.
- SLA rígido de latência e disponibilidade.
- Modelos muito grandes para a infraestrutura disponível.
- Obrigatoriedade de autenticação, auditoria, quotas e observabilidade.
- Necessidade de escalabilidade horizontal com múltiplas GPUs.



Alternativas típicas:

- vLLM para serving de alta performance
- clusters GPU para escala
- gateways multi-modelo para governança
- filas e workers para workloads batch
- observabilidade com métricas e logs

Não force o Ollama local a resolver problema de plataforma de produção.

Exemplo de monitoramento em laboratório

Prática rápida para observar limitações de hardware em diferentes cargas.

- Execute um modelo pequeno e um prompt curto.
- Repita com resposta longa e compare recursos.
- Aumente contexto ou modelo e observe impacto.
- Registre CPU, RAM, VRAM, tempo total e sintomas.
- Conclua qual configuração é viável para a máquina.



Terminal 1: monitoramento

htop

watch -n 1 free -h

watch -n 1 nvidia-smi

Terminal 2: teste

time ollama run llama3.2:3b \

"Explique quantização em 10 tópicos"

O objetivo não é apenas rodar: é entender por que ficou rápido, lento ou instável.

Checklist final de laboratório

Use esta lista antes de considerar o ambiente pronto para aulas ou protótipos.

- Ollama instalado, versão registrada e API respondendo.
- Modelo pequeno baixado, testado e documentado.
- Diretório de modelos e espaço em disco verificados.
- Parâmetros, prompt e tag registrados em matriz de experimento.
- Monitoramento básico de CPU, RAM, VRAM e temperatura validado.

```
resultado_esperado/  
  README.md  
  ambiente.md  
  prompts/  
  modelfiles/  
  resultados.csv  
  evidencias/  
    print_ollama_ps.png  
    nvidia_smi.txt
```

A entrega ideal de um laboratório é reproduzível: qualquer colega consegue repetir o teste.

Referências e fontes de consulta

Fontes usadas para atualização técnica e organização dos exemplos.

- Ollama Docs: API Reference, OpenAI compatibility, Docker, FAQ e Model Library.
- Ollama GitHub: documentação de API, bibliotecas ollama-python e ollama-js.
- Boas práticas gerais de engenharia: registrar modelo, tag, prompt, parâmetros, hardware e métricas.

Sugestão: manter no repositório do curso os Modelfiles, prompts e matrizes de experimentos usados em aula.